

웹 취약점 진단 보고서

Web Application Vulnerability Assessment Report

1. 진단 사이트 개요

진단 호스트	localhost / Target Server (Masked)
종합 진단 결과	취약 발견 (VULNERABLE)

본 보고서는 마스킹 처리된 패킷 데이터를 기반으로 LLM이 진단한 결과 및 N/A 판정 근거를 포함하며, 개발 언어 미특정 환경을 고려하여 주요 프레임워크별 시큐어코딩 가이드를 제공합니다.

2. 진단 결과 요약 및 상세 분석

취약점 명칭	SQL Injection
OWASP 분류	A05 Injection
위험도 (Severity)	HIGH
판단 근거 요약	SQL 구문 오류 메시지가 응답에 포함되어 있으며, 이는 SQL 주입 공격이 성공적으로 수행되었음을 나타냅니다.
요청 분석 근거 (Request)	쿼리 매개변수 'id'에 SQL 주입 패턴 '1' OR '1'='1'이 포함되어 있습니다.
응답 분석 근거 (Response)	응답 본문에 SQL 구문 오류 메시지와 함께 여러 사용자 정보가 노출되었습니다.
보안 영향 (Impact)	SQL 주입을 통해 데이터베이스의 민감한 정보에 접근하거나 데이터베이스를 조작할 수 있습니다.
추가 확인사항 (N/A 전용)	None

3. 프레임워크별 시큐어코딩 및 패치 가이드

대응 방안 요약: SQL 쿼리를 안전하게 처리하기 위해 매개변수화된 쿼리 또는 준비된 문을 사용하십시오.

Spring (Java)

취약한 코드 (Vulnerable)

```
String query = "SELECT * FROM users WHERE user_id = " + userId + "";  
Statement stmt = connection.createStatement();
```

안전한 패치 코드 (Secure)

```
String query = "SELECT * FROM users WHERE user_id = ?";  
PreparedStatement pstmt = connection.prepareStatement(query);  
pstmt.setString(1, userId);
```

Flask (Python)

취약한 코드 (Vulnerable)

```
query = f"SELECT * FROM users WHERE user_id = '{user_id}'"  
cursor.execute(query)
```

안전한 패치 코드 (Secure)

```
query = "SELECT * FROM users WHERE user_id = %s"  
cursor.execute(query, (user_id,))
```

Node.js

취약한 코드 (Vulnerable)

```
const query = `SELECT * FROM users WHERE user_id = '${userId}'`;  
db.query(query);
```

안전한 패치 코드 (Secure)

```
const query = 'SELECT * FROM users WHERE user_id = ?';  
db.query(query, [userId], callback);
```